Reporte técnico: conversión ER → AFND → AFD

Universidad Mariano Gálvez de Guatemala · Teoría de Autómatas · Octubre 2026

Objetivo

Desarrollar un programa que convierta expresiones regulares a autómatas y permita evaluar cadenas sin usar motores de expresiones regulares.

Análisis de la expresión regular

Se lee la expresión de izquierda a derecha y se identifican caracteres, operadores y conjuntos. Los operadores admitidos son concatenación, unión (|), cerradura (*), paréntesis, punto (.), épsilon (ε), conjuntos y sus rangos o negación.

También se aceptan secuencias como \N (dígitos), \A (mayúsculas), \a (minúsculas), \z (letras), \h41 (código hexadecimal) y símbolos escapados. La precedencia es: cerradura, concatenación y unión.

Construcción del AFND con Thompson

Cada símbolo comienza como un autómata pequeño con un inicio, un final y una transición. Para concatenar, se unen fragmentos por épsilon; para la unión, se crean bifurcaciones; para la cerradura, se permiten repeticiones y también cero apariciones.

Los estados del AFND se muestran como q0, q1, q2 y así sucesivamente, junto con las transiciones y estados finales.

Conversión del AFND al AFD

Primero se obtiene la clausura épsilon del estado inicial. Luego se calcula a qué estados se puede llegar al leer cada símbolo y se vuelve a aplicar la clausura épsilon. Cada conjunto nuevo se convierte en un estado del AFD.

Los estados del AFD se nombran Q0, Q1, Q2, etc. Cuando un conjunto contiene el estado final del AFND, se considera de aceptación. Para conjuntos que se superponen, el programa agrupa los caracteres ASCII en clases compatibles.

Evaluación de cadenas

El programa comienza en Q0 y lee la cadena símbolo por símbolo. Muestra el recorrido en el formato Q0|a|Q1. Una cadena se acepta si termina en un estado final; se rechaza si no existe la transición o termina en un estado no final.

El retorno del programa es 1 cuando acepta, 0 cuando rechaza y 2 si los argumentos o la expresión regular son inválidos.

Pruebas y limitaciones

El código tiene pruebas unitarias y casos de ejecución documentados. El alfabeto de trabajo es ASCII (0–127). No se implementan operadores que no están solicitados, como +, ? o repeticiones entre llaves.

La implementación usa Python estándar, sin re ni otros motores regex para detectar patrones.

Referencia

Guía del proyecto ER-AFD, René Ornélyz, Universidad Mariano Gálvez de Guatemala, septiembre de 2026.

Repositorio del proyecto

El código fuente, el Makefile y las pruebas están publicados en GitHub.

https://github.com/cmancillab-creator/ER-AFD